Serrells pendents (grups 11 i/o 23)

4. *Per a cadascun dels algorismes, digueu quin és el temps en cas pitjor quen l'entrada és un enter positiu $n > 0$.*

   Solució: Les vàrem fer totes a classe però vaig dir que donaríem els detalls de l'apartat c) i d).

   (c)

   for $i = 1$ **to** n **do**
       $j = 2$
       while $j < i$ **do**
           $j = j * j$
       end while
   end for

   El bucle exterior fa n iteracions. Per a cada valor de i tenim:

   - un cost constant per a l'assignació $j = i$
   - un bucle intern que fa $argmax\{k \mid 2^{2^k} < i\}$ iteracions.
     Quina és aquesta k?

$$2^{2^k} = i$$
$$2^k = \lg i$$
$$k = \lg\lg i$$

   Per tant, tenim un cost total de:

$$\begin{aligned} T(n) &= n + \textstyle\sum_{i=2}^{n} \lg\lg i \\ &\leq n + \textstyle\sum_{i=2}^{n} \lg\lg n \\ &= \Theta(n \lg\lg n) \end{aligned}$$

   (d)

   i=2
   while $(i * i < n)$ **and** $(n \mod i \neq 0)$ **do**
       $i = i + 1$
   end while

   El pitjor cas es dona quan n és primer i, en aquest cas, es fan $\sqrt{n}$ iteracions. Cada iteració té un cost constant. Per tant, el cost total és $\Theta(\sqrt{n})$.

5. *2SAT.*

   Solució:

   Recordem l'algorisme que vam veure a classe per a comprovar si la fòrmula és satisfactible.

   - Contruir el graf d'implicacions de la instància d'entrada i trobar les seves components fortament connexes.

- Mirar si alguna de les components fortament connexes conté una variable i la seva negació. Si és el cas, aleshores la fòrmula és insatisfactible i acabem. Altrament serà satisfactible.

Faltava veure com fer l'assignació a les variables en cas que la fòrmula sigui satisfactible. Ho farem de la següent forma:

- Construir el graf condensat del graf d'implicacions (el graf que té un vèrtex per cada component fortament connexa, i una aresta de la component i a la component j sempre que el graf original contingui una arc (u,v) tal que u pertany a la component connexa i i v pertany a la component connexa j.
- Ordenar topològicament el graf condensat (noteu que el graf condensat és un graf dirigit acíclic i, per tant, es pot). Podem fer servir l'algorisme de Kosaraju o Tarjan (aquest últim genera l'ordre topològic invers).
- Per a cada component en l'ordre topològic invers, si les seves variables no tenen ja una assignació a TRUE, posar tots els literals a la component a TRUE. Això fa que a tots els literals complementaris d'aquestes a altres components, se'ls hi assigni FALSE. Quan un literal es posa a TRUE, tots els literals que són accessibles a partir d'ell via una cadena d'implicacions també es posaran a TRUE. De manera simètrica, quan un literal és assignat a FALSE, tots els literals que porten a ell via una cadena d'implicacions s'assignaran també a FALSE.

7. *Llisteu les següents funcions en ordre creixent, és a dir, si l'ordre és $f_1; f_2; \ldots$, aleshores $f_2 = \Omega(f_1)$; $f_3 = \Omega(f_2)$; etc.*

$$(\log n)^{100},\ n\log n,\ 3^n,\ \tfrac{n^2}{\log n},\ n2^n,\ 0.99^n,\ n^3,\ \sqrt{n}.$$

Solució:

L'ordre és: 0.99^n; $(\log n)^{100}$; $\sqrt{n}$; $n\log n$; $\frac{n^2}{\log n}$; n^3; $n2^n$; 3^n

10. *Demostreu que $\sum_{j=1}^{\log n} 4^j = O(n^2)$.*

Solució:

$$\sum_{j=1}^{\log n} 4^j \simeq_{(1)} \frac{4^{\log n+1}-1}{4-1} = \frac{4^{\log n+1}-1}{3} = \Theta(4^{\log n}) =_{(2)} O(n^2)$$

Notes:

(1) Progressió geomètrica: $\sum_0^n c^i = \frac{c^{n+1}-1}{c-1}$.
Aproximat perquè no tenim el primer terme ($j=0$).

(2) Per límits: $\lim_{n\to\infty} \frac{n^2}{4^{\log n}} = \infty$

14. *Punt d'articulació.*

Solució: Mireu l'annex.

22. *Majoria de targetes equivalents.*

Solució:

A classe vàrem veure una solució via dividir i véncer que tenia cost $T(n) = O(n \log n)$. Faltava fer una solució de temps lineal. Una possible solució és la següent: emparellem totes les cartes i mirem si les cartes de cada parella són equivalents. Si n és senar, quedarà una targeta sense parella. Per a cada parella de targetes que no són equivalents, descartem les dues targetes. De les parelles que sí són equivalents, en mantenim una de les dues. També mantenim la targeta sense parella en cas que n sigui senar. Anomenarem a aquesta subrutina REDUIR. Aplicarem aquesta subrutina iterativament fins quedar-nos amb una possible targeta candidata, o amb cap.

L'observació que ens porta a un algorisme linial és la següent: Si hi ha una classe d'equivalència amb més de $n/2$ targetes, aleshores la mateixa classe d'equivalència també tindrà més de la meitat de les targetes després de cridar a REDUIR. Això és cert, perquè quan descartem les dues targetes d'una parella, com a mont one d'elles pot pertànyer a la classe d'equivalència majoritària. Una crida a REDUIR sobre un conjunt de n targetes realitza $n/2$ comprovacions d'equivalència i, com a resultat, restaran només $\leq \lceil n/2 \rceil$ targetes. Quan, en repetir aquest procés, ens quedem amb una única targeta, serà la seva classe d'equivalència la candidata a tenir la majoria. Comparem aquesta targeta candidata amb totes les altres per comprovar si, efectivament, la seva classe d'equivalència és majoritària al conjunt de n cartes.

Aquest mètode fa $n/2 + n/4 + \ldots$ comprovacions en les successives aplicacions de REDUIR, més $n-1$ comparacions de parells per a la comprovació final. És a dir, un total de menys de $2n$ comparacions.

Annex

This note reviews some properties of depth first search (DFS) and describes an efficient algorithm for computing the cut vertices in a graph. We restrict our attention to undirected graphs throughout.

1 Depth first search in undirected graphs

The main idea underlying a DFS traversal is very simple:

i) explore edges that lead to unvisited vertices as soon as we stumble upon them;

ii) if we happen to reach a "dead end", we backtrack.

Below is the pseudo-code for the DFS routines.

Algorithm 1 DFS(G)

```
for each vertex u in G do
    mark u as unvisited
    parent[u] = None
time = 0
for each vertex u in G do
    if haven't visited u yet then
        DFS_VISIT(u)
```

Algorithm 2 DFS_VISIT(u)

```
time = time + 1
d[u] = time
mark u as visited
for v ∈ N(u) do
    if haven't visited v yet then
        parent[v] = u
        DFS_VISIT(v)
time = time + 1
f[u] = time
```

DFS traverses the whole input graph, even the graph is not connected. In the process, it generates a DFS forest (defined by the *parent* array) and a *discovery* and *finishing* time for each vertex in the graph. The DFS forest is made up of the following edges

$$\{(u, parent[u]) \mid u \in V \text{ and } parent[u] \neq \text{ None}\} .$$

We will follow the convention that trees in the DFS forest are rooted at the first node discovered in that tree; that is, every time the main DFS routine finds an undiscovered vertex u, a new tree rooted at u is added to the DFS forest T. The forest and the discovery and finishing times have many interesting properties that can be exploited to design algorithms for graph problems.

1.1 Some properties of DFS

Lemma 1 (nesting property)**.** *Let u be a vertex in the graph, and $v_1, \ldots, v_k$ the children of u in the DFS forest. The intervals $[d[v_i], f[v_i]]$ are pair-wise disjoint and span $(d[u], f[v])$.*

Proof. It should be clear from the way the discovery and finishing times are computed: The time is increase just before it is assigned as a discovery time at the beginning of a DFS_VISIT call and just before it is assigned as a finishing time at the end of a DFS_VISIT call. □

Let u and v be two vertices. From the previous lemma, we get that if v is descendant of u then $d[u] < d[v] < f[v] < f[u]$; this follows by applying Lemma 1 repeatedly along the unique u-v path in T. Also, if v is not a descendant of u nor u a descendant of v, the intervals $[d[u], f[u]]$ and $[d[v], f[v]]$ must be disjoint.

Lemma 2 (unvisited path property)**.** *Suppose that right before making the call* DFS_VISIT(u) *there was a path $u \rightsquigarrow v$ consisting solely of undiscovered vertices. Then v will become a descendant of u in the DFS forest.*

Proof. Suppose that was not the case. Then let x be the vertex in the path that is closest to u and is not a descendant of u (we know that there is at least one such vertex (v) so x is well defined) and let y be vertex coming before x in the path (notice that y could be u itself and that y is descendant of u). When we executed DFS_VISIT(y), x was undiscovered and we have the edge (y, x) in the graph. Hence x must be a descendant of y, but y itself is a descendant of u, thus contradicting our assumption. □

It follows that if the graph is connected, the DFS forest is made up of a single tree. The following lemma is an easy consequence of the previous lemma.

Lemma 3 (tree and back edges)**.** *Suppose we run DFS on an undirected graph (V, E). Let (u, v) be some edge in E, then u and v must belong to the same tree in the DFS forest. Furthermore, either u is an ancestor of v, or v is an ancestor of u.*

1.2 Finding cut vertices

Let $G = (V, E)$ be a connected undirected graph. We say a vertex $u \in V$ is a cut vertex if its removal from G disconnects the graph. In this section we study the problem of identifying the set of cut vertices in a given connected graph.

The trivial algorithm iterates over the vertices. In each iteration, we test whether the vertex in question is cut by removing the vertex from the graph and testing the connectivity of the resulting graph (we can use DFS or BFS for that). If we assume adjacency list representation, this algorithm runs in $O(nm)$ time[1]. However, this is not the most efficient algorithm for this problem.

Suppose we run DFS on G and obtain a DFS tree T, and discovery and finishing times for each vertex. This provides some additional information about the graph that we can use to quickly identify cut vertices. The next lemmas assume the graph G is connected.

Lemma 4. *Let r be the root of the DFS tree T; then r is a cut vertex if and only if r has two or more children.*

Proof. If r has a single child, then $T - r$ is still a connected graph. On the other hand, if r has two or more children each children subtree becomes a connected component because the only non-tree edges we are allowed to have are back edges. □

[1]DFS takes $O(n + m)$ time, but because we assume our input graph is connected, the number of edges (m) is at least the number of vertices (n) minus one, so $O(m + n)$ becomes $O(m)$

Lemma 5. *Let u be a leaf in the DFS tree T; then u is no a cut vertex.*

Proof. Since u is a leaf, $T - u$ is still connected. □

To test whether an internal node of T is cut, we need to define one new value:

$$up[u] = \min_{v:(u,v)\in E} d[v].$$

It follows from Lemmas 3 and 1 that the neighbor of u attaining the minimum in the above definition is the neighbor closest to the root. The name $up[]$ is supposed to be a reminder of this. Let us define one last value

$$down\text{-}\&\text{-}up[u] = \min_{w\in T_u} up[w],$$

where T_u is the subtree rooted at u in T.

Lemma 6. *Let u be an internal node of T; then u is cut if and only if u has a child v in T such that*

$$down\text{-}\&\text{-}up[v] = d[u].$$

Proof. Let v be a child of u. After removing u the only way T_v can remain connected to the rest of the graph is through a back edge (x, y) where $x \in T_v$ and y is a strict ancestor of u, but then we would have $down\text{-}\&\text{-}up[v] \leq up[x] \leq d[y] < d[u]$.

On the other hand $down\text{-}\&\text{-}up[v] \leq up[v] \leq d[u]$, where the last inequality follows from the hace that v is connected to u.

□

Lemmas 4, 5, and 6 give an alternative characterization of cut nodes based on the DFS routine, which forms the basis of a more efficient algorithm.

Algorithm 3 FIND-CUT-VERTICES(G)

```
run DFS on G to compute T and d[u] for each u in V
answer = ∅
if root r of T has two or more children then
    add r to answer
for v internal node of T do
    for each child v of u in T do
        if down-&-up[v] = d[u] then
            add u to answer
return answer
```

The correctness of the algorithm rests on the alternative characterization of cut nodes. The running time is dominated by the last `for` loop. For each internal node u, we need to scan every descendant of u, which takes $O(n)$ time. Thus, the running time becomes $O(n^2)$. This is a big improvement from the trivial $O(nm)$ time algorithm, but the algorithm can be further improved. Using some additional ideas, it is possible to get a linear running time of $O(m)$.